

MD5 (Message-Digest Algorithm 5)

Aim

To implement MD5, a cryptographic hash that maps any message to a fixed 128-bit (32 hex character) digest. The message is padded to a multiple of 512 bits, split into blocks, and each block is run through 64 operations (four rounds of sixteen) that mix four 32-bit chaining registers. It is one-way (the input cannot be recovered from the digest) and exhibits the avalanche effect (a one-bit change flips about half the output bits). MD5 is now broken for security use, but remains a clean study of how a hash is built.

Algorithm

MD5 is a one-way digest, so there is no decrypt. The two core methods are padding the message and compressing each block into the running hash. Unlike SHA-1, every word is read and written in little-endian byte order.

```
1 Function Pad(message):
2   bits ← length of message in bits
3   append the byte 0x80 to message
4   while length mod 512 ≠ 448 bits do
5     | append a 0x00 byte
6   append bits as a 64-bit little-endian integer           // now a multiple of 512
7   return message split into 512-bit blocks
```

```
1 Function Digest(message):
2   (a0, b0, c0, d0) ← (67452301, EFCDAB89, 98BADCFE, 10325476)
3   foreach block in Pad(message) do
4     | m[0..15] ← the sixteen 32-bit little-endian words of block
5     | (A, B, C, D) ← (a0, b0, c0, d0)
6     | for i ← 0 to 63 do
7       | (f, g) ← round function and message index for i           // 4 stages of 16
8       | tmp ← B + ((A + f + K[i] + m[g]) rot_left s[i])           // all mod 232
9       | (A, B, C, D) ← (D, tmp, B, C)
10    | a0, b0, c0, d0 ← (a0+A, b0+B, c0+C, d0+D)                   // mod 232
11  return a0 || b0 || c0 || d0                                     // little-endian: 128-bit digest
```

$K[i]$ is a table of 64 constants where $K[i] = \text{floor}(2^{32} * \text{abs}(\sin(i + 1)))$, and $s[i]$ is the per-round left-rotation amount. The per-operation (f, g) come in four stages of sixteen operations:

Operations i	f(b, c, d)	g
0 - 15	(b AND c) OR (NOT b AND d)	i
16 - 31	(d AND b) OR (NOT d AND c)	(5i + 1) mod 16
32 - 47	b XOR c XOR d	(3i + 5) mod 16
48 - 63	c XOR (b OR NOT d)	(7i) mod 16

Output

Digest — message abc (3 bytes)

Padding: append 0x80, fill with zeros, then the 64-bit length $0x18 = 24$ bits written little-endian. The message is short, so it forms a single 512-bit block:

```

61 62 63 80 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 18 00 00 00 00 00 00 00

```

The block is loaded as sixteen little-endian words ($m_0 = 80636261$, $m_1..m_{13} = 0$, $m_{14} = 00000018$, $m_{15} = 0$), then stirred through the 64 operations. The four registers start at the fixed constants and evolve each operation:

Operation i	A	B	C	D
init	67452301	EFCDAB89	98BADCFE	10325476
0	10325476	D6D117B4	EFCDAB89	98BADCFE
1	98BADCFE	344A8432	D6D117B4	EFCDAB89
2	EFCDAB89	2F6FBD72	344A8432	D6D117B4
3	D6D117B4	7AD956F2	2F6FBD72	344A8432
...	...	...	...	...

After 64 operations, the registers are added back into the running hash, giving the final four words:

Word	Value
a0	98500190
b0	B04FD23C
c0	7D3F96D6
d0	727FE128

Serializing each word in little-endian byte order (a0 98500190 becomes bytes 90 01 50 98, and so on) and concatenating gives the 128-bit digest.

Result: `abc -> 900150983cd24fb0d6963f7d28e17f72`

Avalanche — messages `...lazy dog` vs `...lazy cog`

A single-letter change scrambles most of the digest, showing the one-way mixing:

Message	Digest
The quick brown fox jumps over the lazy dog	9e107d9d372bb6826bd81d3542a419d6
The quick brown fox jumps over the lazy cog	1055d3e698d289f2af8663725127bd4b

One changed character flips 71 of the 128 output bits (about 55%). There is no inverse operation: the digest cannot be turned back into the message.